G L O B A L B O O K S

Order Microservice

Technical Documentation & API Reference

Version **1.0.0** | Runtime **Node.js** | Framework **Express 4.x** | Architecture **SOA / Microservice**

Prepared by 22UG2-0004 Lahiru Dilsham

1. Project Overview

The Order Microservice is a standalone REST API built with Node.js and Express, designed as part of the GlobalBooks Service-Oriented Architecture (SOA). It is responsible for the complete lifecycle of a customer book order — from creation and validation through payment processing and shipment dispatch. The service communicates asynchronously with downstream services (Payment and Shipping) using RabbitMQ as a message broker, following an event-driven architecture pattern. All incoming order payloads are validated against a strict JSON Schema using the AJV library, and all write endpoints are protected via JWT Bearer token authentication.

1.1 Key Responsibilities

- Accept and validate incoming order requests via HTTP REST API
- Issue OAuth 2.0-style JWT Bearer tokens for client authentication
- Persist orders in an in-memory store (Map) during the service lifetime
- Publish order events to RabbitMQ for consumption by Payment and Shipping services
- Retrieve individual orders by their generated order ID

1.2 Technology Stack

Component	Technology	Purpose
Runtime	Node.js (CommonJS)	JavaScript execution environment
Framework	Express 4.x	HTTP server and routing
Validation	AJV 8.x + ajv-formats	JSON Schema validation with email/format support
Auth	jsonwebtoken 9.x	JWT token signing and verification
Messaging	amqplib 0.10.x	RabbitMQ AMQP client for event publishing
Dev Server	nodemon 3.x	File-watching auto-restart in development

2. Project Structure

The service follows a clean, layered directory structure that separates concerns: routing, middleware, configuration, business logic, schema validation, and messaging setup each live in their own directory.

```
order_service/
├── index.js              # Application entry point
├── package.json          # Dependencies and scripts
├── jsconfig.json         # JS path alias configuration
│
├── config/
│   └── constant.js       # Centralized constants (PORT, secrets)
│
├── routes/
│   ├── auth_router.js    # POST /oauth/token — token issuance
│   └── orders.js         # POST /orders | GET /orders/:id
│
├── middlewares/
│   └── authentication.js # JWT Bearer token verification middleware
│
├── schema/
│   └── orderSchema.json  # AJV JSON Schema for order payloads
│
├── services/
│   ├── payment_service.js  # Standalone consumer: payments.process queue
│   └── shipping_service.js # Standalone consumer: shipping.dispatch queue
│
└── Integration/
    └── setupRabbitMQ.js  # One-time RabbitMQ topology provisioner
```

2.1 File Responsibilities

index.js: Bootstraps the Express application, registers global middleware (JSON body parser), mounts routers, and starts the HTTP listener on the configured port.

config/constant.js: Single source of truth for all magic values — JWT secret, OAuth client credentials, error messages, and server port. Modify this file to reconfigure the service without touching business logic.

routes/auth_router.js: Handles the OAuth 2.0 client credentials grant. Accepts client_id and client_secret, issues a signed JWT valid for 1 hour upon successful authentication.

routes/orders.js: Core order business logic. Validates payloads against JSON Schema, creates orders with auto-generated IDs, publishes events to RabbitMQ, and retrieves orders by ID.

middlewares/authentication.js: Express middleware that extracts and verifies the Bearer JWT from the Authorization header before allowing access to protected routes.

schema/orderSchema.json: Strict AJV JSON Schema defining required fields, data types, and format constraints for incoming order payloads.

services/payment_service.js: Standalone RabbitMQ consumer for the payments.process queue. Simulates payment processing and publishes payment.completed events on success, or routes to the Dead Letter Queue on failure.

services/shipping_service.js: Standalone RabbitMQ consumer for the shipping.dispatch queue. Simulates shipment dispatch upon receiving payment.completed events.

Integration/setupRabbitMQ.js: One-time setup script that provisions the entire RabbitMQ topology: exchanges, queues, bindings, and Dead Letter Queue. Run this once before starting the service.

3. Installation & Setup

3.1 Prerequisites

Ensure the following are installed and running on your machine before proceeding:

- Node.js v18+ (LTS recommended)
- npm v9+
- RabbitMQ server running on localhost:5672 (default AMQP port)
- Git (for cloning the repository)

> ℹ RabbitMQ must be running before you start the service. If it is not available, the service will still start but event publishing will silently fail.

3.2 Step-by-Step Installation

1. Clone the repository or extract the project archive into your working directory.
2. Navigate into the project root directory:

```
cd order_service
```

3. Install all npm dependencies:

```
npm install
```

4. Provision the RabbitMQ topology (run once, before first start):

```
node Integration/setupRabbitMQ.js
```

5. Start the development server:

```
npm start
```

You should see the following console output confirming the service is ready:

```
OrdersService listening on port 3000
RabbitMQ Connected & Channel Ready
```

3.3 Configuration

All configurable values live in config/constant.js. The defaults are shown below:

```
// config/constant.js
const TOKEN        = 'secret_key';          // JWT signing secret
const CLIENT_ID    = 'globalbooks_client';  // OAuth client identifier
const CLIENT_SECRET = 'secret123';          // OAuth client secret
const PORT         = 3000;                  // HTTP server port
```

ℹ Production deployments should replace hardcoded secrets with environment variables (e.g., process.env.JWT_SECRET). Never commit real credentials to source control.

4. API Reference

The service exposes three HTTP endpoints across two routers mounted at different path prefixes. The base URL when running locally is http://localhost:3000.

4.1 Endpoint Summary

Method	Path	Description	Auth
GET	/	Health check — confirms service is running	Public
POST	/oauth/token	Request a Bearer access token	Public
POST	/orders	Create a new order (validates payload via AJV)	Public
GET	/orders/:id	Retrieve an existing order by its generated ID	Public

ℹ Note: The authentication middleware (authenticateToken) is imported in index.js but is not currently applied to the order routes. To protect the /orders endpoints, add it as route-level middleware.

4.2 GET / — Health Check

Endpoint	GET http://localhost:3000/
Auth Required	No
Content-Type	application/json

Response — 200 OK

```
{
 "message": "Order Service is Running!"
}
```

4.3 POST /oauth/token — Request Access Token

This endpoint implements an OAuth 2.0 Client Credentials Grant. Submit your client_id and client_secret to receive a signed JWT Bearer token. The token expires after 1 hour (3600 seconds) and must be included in the Authorization header for protected requests.

Endpoint	POST http://localhost:3000/oauth/token
Auth Required	No — public endpoint
Content-Type	application/json

Request Body

```
{
 "client_id":     "globalbooks_client",
 "client_secret": "secret123"
}
```

Response — 200 OK (Token Issued)

```
{
 "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
 "token_type":   "Bearer",
 "expires_in":   3600
}
```

Response — 401 Unauthorized (Invalid Credentials)

```
{
 "error": "invalid_client"
}
```

Usage — Subsequent Authenticated Requests

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

4.4 POST /orders — Create a New Order

Creates a new book order. The request body is validated against the JSON Schema defined in schema/orderSchema.json using AJV. If validation passes, a new order is created with an auto-generated ID, stored in memory, and an order.created event is published to RabbitMQ.

Endpoint	POST http://localhost:3000/orders
Auth Required	No (middleware available but not currently applied)
Content-Type	application/json

Request Body Schema

Field	Type	Required	Constraints
bookIsbn	string	Yes	ISBN string, e.g. "978-0132350884"
quantity	integer	Yes	Minimum value: 1
customerEmail	string	Yes	Must be a valid email format

Request Body (Valid Example)

```
{
 "bookIsbn":     "978-0132350884",
```

```
  "quantity":      3,
  "customerEmail": "dev_expert@example.com"
}
```

Response — 201 Created

> ℹ The server auto-generates the id (ORD-{timestamp}), status (PENDING_PAYMENT), and createdAt fields.

```
{
  "id":           "ORD-1715634821000",
  "bookIsbn":      "978-0132350884",
  "quantity":      3,
  "customerEmail": "dev_expert@example.com",
  "status":        "PENDING_PAYMENT",
  "createdAt":     "2026-03-23T15:05:00.000Z"
}
```

Response — 400 Bad Request (Schema Validation Failure)

AJV will return detailed error messages for each failing field. The example below shows what happens when quantity is sent as a string and customerEmail is malformed:

```
{
  "error": "Invalid Request Payload",
  "details": [
    {
      "instancePath": "/quantity",
      "schemaPath":   "#/properties/quantity/type",
      "keyword":      "type",
      "params":       { "type": "number" },
      "message":      "must be number"
    },
    {
      "instancePath": "/customerEmail",
      "schemaPath":   "#/properties/customerEmail/format",
      "keyword":      "format",
      "params":       { "format": "email" },
      "message":      "must match format \"email\""
    }
  ]
}
```

4.5 GET /orders/:id — Retrieve an Order

Retrieves a specific order from the in-memory store using its order ID. The ID is passed as a URL path parameter.

Endpoint	GET http://localhost:3000/orders/:id
Path Parameter	:id — The order ID, e.g. ORD-1715634821000
Auth Required	No (middleware available but not currently applied)

Response — 200 OK

```
{
  "id":           "ORD-1715634821000",
  "bookIsbn":      "978-0132350884",
  "quantity":      3,
  "customerEmail": "dev_expert@example.com",
  "status":        "PENDING_PAYMENT",
  "createdAt":     "2026-03-23T15:05:00.000Z"
}
```

Response — 404 Not Found

```
{
  "error": "Order not found"
}
```

5. Authentication & Authorization

The service uses a lightweight OAuth 2.0 Client Credentials flow backed by JSON Web Tokens (JWT). All token operations use the HS256 algorithm (HMAC with SHA-256).

5.1 Token Acquisition Flow

6. The client sends a POST request to /oauth/token with valid client credentials.
7. The auth router validates client_id and client_secret against the constants in config/constant.js.
8. On success, jwt.sign() creates a signed token embedding { clientId } as the payload, signed with CLIENT_SECRET, expiring in 1 hour.
9. The token is returned as { access_token, token_type: 'Bearer', expires_in: 3600 }.
10. The client includes the token in all subsequent requests: Authorization: Bearer <token>.

5.2 Middleware — authenticateToken

The authentication middleware in middlewares/authentication.js performs the following checks on every request:

- Reads the Authorization header and verifies it starts with 'Bearer '.
- Extracts the token string and calls jwt.verify() using the TOKEN constant.
- Returns 401 Unauthorized if the header is missing or malformed.
- Returns 403 Forbidden if the token is expired, tampered, or invalid.
- Attaches the decoded payload to req.user and calls next() on success.

> **i** The middleware is defined and exported but not yet applied to the /orders routes. Apply it in index.js or within the orders router to enforce authentication on order endpoints.

```
// To apply globally in index.js:
app.use('/orders', authenticateToken, ordersRouter);


// Or per-route in routes/orders.js:
router.post('/', authenticateToken, async (req, res) => { ... });
```

6. RabbitMQ Event Architecture

The service uses RabbitMQ to decouple order creation from payment processing and shipment dispatch. It follows an event-driven pub/sub model using a topic exchange, allowing flexible routing based on binding keys.

6.1 Topology Overview

```
┌──────────────────────────────────────────────────────────────────────────────────────┐
│            RabbitMQ Topology                                                          │
├──────────────────────────────────────────────────────────────────────────────────────┤
│                                                                                       │
│                                                                                       │
│  [Order Service]  ──order.created──▶  globalbooks.events                              │
│                                  (topic exchange)                                     │
│                                    │          │          │                             │
│                            order.created      payment.compl                           │
│                                  ▼                ▼                                   │
│                        payments.process     shipping.dispatch                         │
│                                  │                │                                   │
│                      [Payment Service]    [Shipping Service]                           │
│                                  │                │                                   │
│                      On failure: ──error──▶  globalbooks.dlx                          │
│                                        (DLX exchange)                                 │
│                                            │          │                               │
│                                      globalbooks.dlq                                  │
│                                                                                       │
└──────────────────────────────────────────────────────────────────────────────────────┘
```

6.2 Exchanges, Queues & Bindings

Name	Type	Binding Key	Consumer
globalbooks.events	topic	—	Main exchange (publisher)
payments.process	queue	order.created	payment_service.js
shipping.dispatch	queue	payment.completed	shipping_service.js
globalbooks.dlx	direct	error	Dead Letter Exchange
globalbooks.dlq	queue	error	Manual inspection queue

6.3 Order Lifecycle (Event Flow)

11. Client creates an order via POST /orders.
12. Order Service publishes an order.created event to globalbooks.events.
13. RabbitMQ routes the event to the payments.process queue (bound to order.created).
14. Payment Service consumes the event, simulates processing (1s delay), then either:

- Success (90% probability): publishes payment.completed to globalbooks.events.

- Failure (10% probability): nacks the message, routing it to globalbooks.dlq via the DLX.

15. On payment.completed, the shipping.dispatch queue receives the event.
16. Shipping Service consumes it, simulates dispatch (1s delay), and logs confirmation.

6.4 RabbitMQ Channel Optimization

The orders.js route file maintains a single persistent RabbitMQ channel (rabbitChannel) initialized at startup via setupRabbitMQ(). This avoids the overhead of creating a new connection and channel for each HTTP request, which would be a serious performance bottleneck in high-throughput scenarios.

```
// Single channel created at startup
let rabbitChannel = null;
async function setupRabbitMQ() {
  const conn = await amqp.connect('amqp://localhost');
  rabbitChannel = await conn.createChannel();
  await rabbitChannel.assertExchange('globalbooks.events', 'topic', { durable: true });
}
setupRabbitMQ(); // Called once on module load
```

ℹ If RabbitMQ is unavailable at startup, rabbitChannel remains null. The service will still process orders but will skip event publishing, which means Payment and Shipping services will not be notified.

7. JSON Schema Validation

All POST /orders request bodies are validated using AJV (Another JSON Validator) v8 with ajv-formats for extended format support (including the 'email' format). The schema is loaded from schema/orderSchema.json.

7.1 Order Schema Definition

```
{
 "type": "object",
 "properties": {
  "bookIsbn":      { "type": "string" },
  "quantity":      { "type": "integer", "minimum": 1 },
  "customerEmail": { "type": "string",  "format": "email" }
 },
 "required":            ["bookIsbn", "quantity", "customerEmail"],
 "additionalProperties":  false
}
```

7.2 Validation Rules

Field	Rule	Notes
bookIsbn	type: string	Any string accepted; consider adding pattern for strict ISBN validation
quantity	integer, min: 1	Must be a whole number ≥ 1; floats and strings are rejected
customerEmail	string, format: email	Enforced by ajv-formats; malformed emails return a 400 error

Field	Rule	Notes
(any other field)	additionalProperties: false	Any unrecognized fields cause a validation failure

7.3 Validation Error Format

AJV returns a structured array of errors in the details field of a 400 response. Each error object contains:

- instancePath — JSON pointer to the failing field in the request body
- schemaPath — pointer to the failing rule in the schema
- keyword — the constraint keyword that failed (e.g., type, format, required)
- params — additional parameters describing the failure
- message — human-readable error message

8. API Testing Guide

Use any HTTP client to test the endpoints. The examples below use cURL, but you can use Postman, Insomnia, or any REST client that supports setting request headers and JSON bodies.

8.1 Step 1 — Verify Service is Running

```
curl http://localhost:3000/

# Expected response:
# { "message": "Order Service is Running!" }
```

8.2 Step 2 — Obtain an Access Token

```
curl -X POST http://localhost:3000/oauth/token \
  -H "Content-Type: application/json" \
  -d '{"client_id": "globalbooks_client", "client_secret": "secret123"}'

# Expected response:
# { "access_token": "eyJhb...", "token_type": "Bearer", "expires_in": 3600 }
```

8.3 Step 3 — Create an Order

```
curl -X POST http://localhost:3000/orders \
  -H "Content-Type: application/json" \
  -d '{"bookIsbn": "978-0132350884", "quantity": 3, "customerEmail": "dev@example.com"}'

# Expected response (201 Created):
# { "id": "ORD-...", "status": "PENDING_PAYMENT", ... }
```

8.4 Step 4 — Retrieve the Order

```
# Replace ORD-1715634821000 with the id from Step 3
curl http://localhost:3000/orders/ORD-1715634821000
```

```
# Expected response (200 OK):
# { "id": "ORD-1715634821000", "status": "PENDING_PAYMENT", ... }
```

8.5 Step 5 — Test Validation Error

```
curl -X POST http://localhost:3000/orders \
  -H "Content-Type: application/json" \
  -d '{"bookIsbn": "978-0132350884", "quantity": "five", "customerEmail": "not-an-email"}'

# Expected response (400 Bad Request):
# { "error": "Invalid Request Payload", "details": [ ... ] }
```

8.6 Step 6 — Test 404 Not Found

```
curl http://localhost:3000/orders/ORD-0000000000000

# Expected response (404 Not Found):
# { "error": "Order not found" }
```

9. Error Reference

Status	Error Code	Cause	Endpoint
400	Invalid Request Payload	AJV schema validation failed; details array included	POST /orders
401	Unauthorized User	Authorization header missing or does not start with 'Bearer '	Protected routes
401	invalid_client	Wrong client_id or client_secret submitted	POST /oauth/token
403	Forbidden	JWT token expired, tampered with, or signed with wrong secret	Protected routes
404	Order not found	No order with the provided :id exists in the in-memory store	GET /orders/:id

10. Known Limitations & Recommendations

The following items represent known limitations of the current implementation and recommended improvements for a production-grade deployment:

10.1 In-Memory Storage

Orders are stored in a JavaScript Map (ordersDb), which is process-scoped and non-persistent. All data is lost when the service restarts. For production, replace with a persistent database such as PostgreSQL or MongoDB.

10.2 Missing Route-Level Authentication

The authenticateToken middleware is implemented but not applied to the /orders routes in the current codebase. Apply it to enforce authorization on order creation and retrieval endpoints.

10.3 Hardcoded Secrets

JWT secrets and OAuth client credentials are hardcoded in config/constant.js. Production deployments must use environment variables (process.env) and a secrets management solution such as AWS Secrets Manager or HashiCorp Vault.

10.4 No RabbitMQ Reconnection Logic

If RabbitMQ drops the connection after startup, rabbitChannel becomes stale and publish calls will silently fail. Implement a reconnection strategy (e.g., exponential backoff with amqp-connection-manager) for production reliability.

10.5 Order ID Collision Risk

Order IDs are generated using Date.now() (millisecond timestamp). Under high concurrency, two orders created within the same millisecond would have colliding IDs. Use a UUID library (e.g., uuid v4) for globally unique, collision-free identifiers.

10.6 Missing Input Sanitization

While AJV validates field types and formats, there is no sanitization of string content (e.g., for SQL injection or XSS). Add a sanitization layer if order fields are persisted or rendered in a UI.

10.7 No Pagination for Order Listing

The service does not expose a GET /orders endpoint to list all orders. If added in the future, implement pagination to avoid unbounded responses.

11. Dependency Reference

Package	Version	Role
express	^4.18.2	HTTP server, routing, and middleware framework
ajv	^8.12.0	JSON Schema validator (core engine)
ajv-formats	^2.1.1	Adds email, URI, date-time format validators to AJV
jsonwebtoken	^9.0.0	JWT token signing (HS256) and verification
amqplib	^0.10.9	AMQP 0-9-1 client for RabbitMQ communication
nodemon	^3.1.14	Development: auto-restarts server on file changes